— *Solution notes* —

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 4

2 Programming in C and C++ (djg11)

pointers and arrays

(*a*) Arrays in C are accessed using square bracket notation.

(*i*) What are the advantages and disadvantages of array bounds checking?

Answer: Array bounds checking should not be required in working code, it would only slow it down. But better security arises from subscript checking, both against malicious code injection and programmer errors.

(*ii*) Define with examples an array access and a pointer dereference in the C language explaining the underlying equivalents.

Answer: Array access uses `a[s]` which is shorthand for the pointer dereference `*(a+b)` which, oddly, makes the array subscription construct commutative owing to the nature of pointer addition: pointer addition requires an integer and a pointer type, but is not fussy which.

(*iii*) State all similarities between array access and pointer dereference and comment on interactions with possible bounds checks.

[5 marks]

Answer:

By allowing array subscription with arbitrary pointers, a given pointer is not tied to a particular array. This provides greater expressivity (e.g. it can point to a scalar head var in linked-list enqueue) but this makes bounds checking meaningless.

A common programming idiom is to step a pointer over an array with a `for` loop. This has an out-of-bounds error on loop exit that must not be flagged with a run-time error, but would not be if bounds checks are only made during derefernces.

---

undefined behaviour

(*b*) What advantage is gained from allowing a given C program to vary in execution behaviour from one computer architecture to another? Give three common example variations. What is the disadvantage of variation? [5 marks]

Answer: Primary advantages are efficiency arising from not having to respect uniformity and machine-specific low-level access. Three principle examples are sizeof applied to 32 and 64 bit pointers, big v little endian and stack direction which can often also influence order of operand evaluation. Of course there are other variations that just depend on the libraries or O/S installed on a machine, such as scheduling order for threads and comparison predicates applied to successive `malloc` results, but programs should work in all cases. Main disadvantage is code may be tested on one machine but not written in a sufficiently portable style and so fails unexpectedly on another.

C++ templates

(*c*) Give two ways in which C++ templates differ from Java Generics, other than mere syntactic differences. [4 marks]

Answer: Exact syntax is not important so long as the ideas are clear. Java Generics require parameters to be Java reference types. In C++ parameters may be of other types, e.g. integers, compare:

```
template<class C, int n> struct Buffer { C[n] buf; int size; }
class Buffer<C> { public C[] buf; }
```

C++ templates may be *specialised*. Here MyArray just behaves like arrays for most types, but for type `bool` it uses a bit-map representation. Note that the access

```
template<class C, int n> struct MyArray { C[n] buf; }
template<int n> struct MyArray<bool,n> { unsigned char[n/8] bitmap;}
```

Note: the getter and setter for elements would also need to be specified, perhaps by overloading `[]` but this is beyond the example.

An additional difference (perfectly valid in an answer) is that Java generics are implemented with 'type erasure' (once in the generated `.class` file) with casts to/from `object`, while C++ templates are implemented by textual expansion for each distinct template instance appearing in the program—similarly to macros.

C++ virtual

(*d*) Giving a suitable example, explain the effect in C++ of qualifying a member function (method) with `virtual`. [3 marks]

Answer: By default C++ uses the compile-time type of an object to determine which member function to invoke, using `virtual` causes it to use the run-time type. Hence given

```
class A
{   const int a = 1
    public:
    int f() { return a; }
    virtual int g() { return a; }
}
class B : A
{   const int a = 2;
    public:
    int f() { return a; }
    virtual int g() { return a; }
}
```

Then `((A *)new B)->f()` evaluates to 1, but `((A *)new B)->g()` evaluates to 2.

C++ RAII

(*e*) Recode the following Java code in C++. Minor syntactic errors will not be penalised.

```
class Foo
{   final int[] v; final int s;
    public Foo(int n) { s = n; v = new int[n]; }
    // In Java garbage collection de-allocates arrays
    // appearing in no-longer-used instances of Foo.
    // In C++ an alternative solution is required.
}
```

[3 marks]

Answer:

```
class Foo
{  int *const v; const int s;
   public:
   Foo (int n) : s(n), v(new int[n]) {}
   virtual ~Foo() { delete[] v; }
}
```

Marks for: (*i*) correct placement of const and for initialisation not assignment; (*ii*) defining a destructor (using 'virtual' is best-practice, but not required for full marks); (*iii*) using `delete[]`.